

```

import numpy as np
import matplotlib.pyplot as plt
import csv

def load_data(file_path):
    edges = []
    with open(file_path, 'r') as file:
        reader = csv.reader(file)
        for row in reader:
            source, target, rating, _ = map(float, row)
            edges.append((int(source), int(target), rating)) # Rating is the weight
    return edges

def create_adjacency_matrix(edges):
    nodes = set()
    for source, target, _ in edges:
        nodes.update([source, target])
    node_list = sorted(nodes)
    node_index = {node: idx for idx, node in enumerate(node_list)}
    size = len(node_list)
    adj_matrix = np.zeros((size, size))
    for source, target, weight in edges:
        i, j = node_index[source], node_index[target]
        adj_matrix[i, j] = weight
    return adj_matrix, node_list

edges = load_data("soc-sign-bitcoinalpha.csv")
adj_matrix, node_list = create_adjacency_matrix(edges)

```

✓ Degree and Clustering Coefficient in Weighted Directed Networks

1. Degree in a Weighted Directed Network

In a **weighted directed graph**, each edge has a direction and an associated weight. The degree of a node is computed by considering both **incoming** and **outgoing** edges.

- **Weighted In-degree (k_{in}^i):**
The sum of weights of all incoming edges to node (i).
$$k_{in}^i = \sum_j w_{ji}$$
 where (w_{ji}) is the weight of the edge from node (j) to node (i).
- **Weighted Out-degree (k_{out}^i):**
The sum of weights of all outgoing edges from node (i).
$$k_{out}^i = \sum_j w_{ij}$$
 where (w_{ij}) is the weight of the edge from node (i) to node (j).
- **Total Weighted Degree (k_i):**
The sum of in-degree and out-degree.
$$k_i = k_{in}^i + k_{out}^i$$

2. Clustering Coefficient in a Weighted Directed Network

The **weighted clustering coefficient** measures the likelihood that two neighbors of a node are connected, taking into account the edge weights.

The weighted clustering coefficient for a node (i) is given by:

$$C_i = \frac{k_i(k_i - 1)}{\sum_{j,k} \left(\frac{w_{ij} + w_{ik}}{2} \right) w_{jk}}$$

where:

- (k_i) is the total degree of node (i).
- (w_{ij}) and (w_{ik}) are the weights of edges from node (i) to nodes (j) and (k).
- (w_{jk}) is the weight of the edge between nodes (j) and (k).

```

def calculate_degrees(adj_matrix):
    in_degree = np.sum(adj_matrix, axis=0)
    out_degree = np.sum(adj_matrix, axis=1)
    total_degree = in_degree + out_degree
    return in_degree, out_degree, total_degree

```

```

def calculate_weighted_clustering_coefficient(adj_matrix):
    n = adj_matrix.shape[0]
    clustering_coeffs = np.zeros(n)
    for i in range(n):
        neighbors = np.nonzero(adj_matrix[i, :] + adj_matrix[:, i])[0]
        k_i = len(neighbors)
        if k_i < 2:
            continue
        sum_weights = 0.0
        for j in neighbors:
            for k in neighbors:
                if j != k:
                    sum_weights += (adj_matrix[i, j] + adj_matrix[i, k]) * adj_matrix[j, k]
        clustering_coeffs[i] = sum_weights / (k_i * (k_i - 1))
    return clustering_coeffs

in_deg, out_deg, total_deg = calculate_degrees(adj_matrix)
clustering_coeffs = calculate_weighted_clustering_coefficient(adj_matrix)

def plot_weighted_degree_distribution(degrees, filename="degree_distribution.png"):
    plt.figure()
    plt.hist(degrees, bins=20, edgecolor='black', alpha=0.75)
    plt.xlabel("Weighted Degree")
    plt.ylabel("Frequency")
    plt.title("Weighted Degree Distribution")
    plt.grid()
    plt.savefig(filename)
    plt.close()

plot_weighted_degree_distribution(total_deg)

def plot_clustering_vs_degree(degrees, clustering_coeffs, filename="clustering_vs_degree.png"):
    plt.figure()
    plt.scatter(degrees, clustering_coeffs, alpha=0.75)
    plt.xlabel("Weighted Degree")
    plt.ylabel("Clustering Coefficient")
    plt.title("Clustering Coefficient vs Degree")
    plt.grid()
    plt.savefig(filename)
    plt.close()

plot_clustering_vs_degree(total_deg, clustering_coeffs)

```